

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK
FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM3522 PROJE1 ve PROJE2 RAPORU

Proje1: Çift Katmanlı Web Uygulaması

https://github.com/aybaranyurtseven/BLM3522_Project1

<https://youtu.be/fVFFJlhwy5M>

Proje2: Gerçek Zamanlı Veri Akışı ve İşleme

https://github.com/aybaranyurtseven/BLM3522_Project2

https://youtu.be/_6byVcgBJD4

Aybaran Yurtseven – 22290067

Nisan 2026

İçindekiler

I	Proje 1: Hasta Takip Sistemi	2
1	Proje Hakkında	2
2	Kullanılan Teknolojiler	2
2.1	Backend (API Katmanı)	2
2.2	Frontend (Kullanıcı Arayüzü)	2
2.3	Bulut Çözümleri (Cloud Platform)	2
3	Proje Geliştirme Süreci ve Adımlar	2
4	Uygulamayı Çalıştırma (Lokal)	3
5	Sonuç ve Kazanımlar	3
II	Proje 2: Gerçek Zamanlı Veri Akışı ve İşleme	4
6	Projenin Amacı ve Kapsamı	4
7	Kullanılan Teknolojiler	4
8	Sistem Mimarisinin Detayları	4
8.1	Veri Üretici (IoT Simülatörü)	4
8.2	Veri Tüketici (Cloud Processor)	4
9	Test ve Doğrulama	4

Kısım I

Proje 1: Hasta Takip Sistemi

1 Proje Hakkında

Bu proje, Bulut Bilişim (3522) dersi kapsamında geliştirilen çift katmanlı (Web API ve Frontend) bir web uygulamasıdır. Temel amacı hastaların kaydedilmesi, bilgilerinin güncellenmesi ve takiplerinin yapılmasıdır. Proje RESTful API mimarisi kullanılarak iki ayrı katman halinde (Backend ve Frontend) modüller şeklinde inşa edilmiştir.

Uygulamanın veritabanı lokal bir sunucuda değil, doğrudan AWS RDS (Relational Database Service) kullanılarak bulutta konumlandırılmıştır.

2 Kullanılan Teknolojiler

2.1 Backend (API Katmanı)

- **C# / ASP.NET Core:** Temel programlama dili ve framework.
- **Entity Framework Core:** SQL Server bağlantısı ve Code-First (migration) yaklaşımı.
- **Swagger UI / Scalar:** API uç noktalarının dokümantasyonu ve test edilmesi.

2.2 Frontend (Kullanıcı Arayüzü)

- **React.js:** Modern ve hızlı kullanıcı arayüzü geliştirme altyapısı.
- **Axios:** Backend API'sine asenkron HTTP istekleri atmak için kullanıldı.

2.3 Bulut Çözümleri (Cloud Platform)

- **AWS RDS (SQL Server):** Veritabanının bulut üzerinde barındırılması ve canlı veri yönetimi.

3 Proje Geliştirme Süreci ve Adımlar

Ödev gereksinimlerine uygun olarak projenin yapımı belirli aşamalara ayrılmıştır:

1. **Git Deposu Kurulumu:** Kodların versiyon kontrolü ve takibi için projenin bulunduğu dizinde lokal git deposu oluşturulmuştur.
2. **Uygulama Mimarisi Planlaması:** Uygulama `HastaTakip.API` ve `hasta-takip-ui` olarak iki ana modüle ayrılmıştır.
3. **AWS RDS Kurulumu:** Amazon Web Services üzerinde SQL Server altyapısı ayağa kaldırılmış ve sadece uygulamanın IP'sinden (veya izin verilen ağlardan) erişilebilecek dış bağlantı ayarları yapılmıştır.

4. Backend Geliřtirmesi:

- ASP.NET Core ile RESTful API altyapısı kurulmuřtur.
 - Entity Framework ile AWS üzerindeki veritabanına Code-First ile tablolar aktarılmıřtır.
 - Frontend uygulamasının erişimi için CORS politikaları düzenlenmiştir.
5. **Frontend Geliřtirmesi:** *create-react-app* ile React řablonu oluřturulmuř, ardından Axios ile API'ye baėlanarak veritabanından alınan hasta listeleri gösterilmiř ve veri giriř ekranları tasarlanmıřtır.
6. **Test ve Entegrasyon:** Uygulama komut satırı üzerinden (`dotnet run` ve `npm start`) test edilmiř, arayüzün sorunsuz řekilde DB'ye kayıt yapabildiėi teyit edilmiřtir.

4 Uygulamayı Çalıştırma (Lokal)

Projeyi çalıştırmak için backend ve frontend servislerinin aynı anda çalışması gerekmektedir.

- **Backend:** `HastaTakip` API dizinine girilerek `dotnet run` komutu çalıştırılır (Swagger/Scalar dokümantasyonu için: <http://localhost:5085/scalar/v1>).
- **Frontend:** Yeni bir terminalde `hasta-takip-ui` dizinine girilerek `npm start` komutuyla proje ayaėa kaldırılır (<http://localhost:3000>).

5 Sonuç ve Kazanımlar

Bu proje kapsamında; bir uygulamanın uçtan uca (frontend, backend ve bulut veritabanı) nasıl ayaėa kaldırılacaėı, bulut biliřim hizmetlerinin modern web geliřtirmeye entegrasyonu (AWS RDS) ve REST API - Client iletiřimi gibi kritik konular bařarıyla öğrenilmiř ve uygulanmıřtır.

Kısım II

Proje 2: Gerçek Zamanlı Veri Akışı ve İşleme

6 Projenin Amacı ve Kapsamı

Bu projenin temel amacı, IoT aygıtlarından gelen sensör verilerinin simüle edilerek bulut ortamına (AWS) gönderilmesi ve akabinde gerçek zamanlı olarak okunarak veritabanına aktarılmasıdır. Sistem C# (.NET 8) mimarisi ile tasarlanmış, AWS platformundan Kinesis servisi kullanılmıştır.

7 Kullanılan Teknolojiler

- **Programlama Dili:** C# (.NET 8)
- **Bulut Akış Servisi:** AWS Kinesis Data Streams
- **Hedef Veritabanı:** AWS (Database: `database1`) - SQL Server
- **Kütüphaneler:** AWSSDK.Kinesis, Microsoft.Data.SqlClient, System.Text.Json

8 Sistem Mimarisinin Detayları

Sistem 2 temel bölümden oluşmaktadır.

8.1 Veri Üretici (IoT Simülatörü)

Bir *Console Application* projesi olan `IoT_Simulator`, belirli bir cihaz ID'si ve mantıksal sınırlarla rastgele sıcaklık, nem verileri üretir. Saniyede 1 kere json formuna çevirdiği metni AWS Kinesis üzerinden `SensorDataStream` kuyruğuna bırakır.

8.2 Veri Tüketici (Cloud Processor)

`Cloud_Processor` uygulaması Kinesis akışındaki verileri saniyede bir kez (polling yöntemiyle) denetler. Yeni bir kayıt düştüğünde `Deserialize` yaparak alanları okur, en son aşamada AWS'teki `database1` adlı SQL tablosuna kaydeder.

9 Test ve Doğrulama

Proje AWS yapısı tam olarak hazır olmasa bile (bağlantı bilgileri girilmeden çalıştırıldığında) exception fırlatmak yerine uyarı verip *Simülasyon Moduna* girer. Bu sayede çalışması videoya çekilirken konsoldaki uyarı logları ile tüm akış açıklanabilir durumdadır. Tüm çalışma düzenli `git commit` yapısı ile korunmaktadır.